

Social Seller — Guide Sécurité

Emails par environnement · Bonnes pratiques · Prompts Claude Code sécurisés

Ce guide couvre trois axes : la gestion des emails par environnement, les règles de sécurité fondamentales pour une app SaaS multi-tenant, et les instructions à donner systématiquement à Claude Code pour générer du code sécurisé.

Partie 1 — Emails par Environnement

En SaaS, chaque environnement doit avoir des emails distincts et isolés pour éviter les accidents (envoyer un vrai email à un vrai client depuis l'environnement de test).

Les 3 environnements à distinguer

Environnement	Rôle	Emails recommandés	Règle absolue
■ Development (local)	Tu codes et testes sur ta machine	noreply@socialseller.app noreply-dev@socialseller.app	Ne jamais envoyer de vrais emails. Utilise Mailpit ou Mailtrap.
■ Staging (pré-prod)	Tests avant mise en production	staging@socialseller.app noreply-staging@socialseller.app	Emails envoyés uniquement à l'équipe interne. Jamais aux clients.
■ Production	L'app réelle utilisée par tes clients	clients@socialseller.app support@socialseller.app admin@socialseller.app	Seul environnement qui envoie aux vrais utilisateurs.

Emails de service à créer (Production)

Email	Usage	Pourquoi séparé ?
noreply@socialseller.app	Activations, rappels, notifications auto	L'utilisateur ne doit pas répondre à cet email
support@socialseller.app	Support client, contact humain	Séparé pour trier les vraies demandes
admin@socialseller.app	Super Admin — alertes système internes	Jamais exposé aux clients
billing@socialseller.app	Factures, renouvellements (futur)	Isolé pour conformité comptable
no-reply@socialseller.app	Alias de noreply si nécessaire	Redirection vers noreply

Outil recommandé en développement : Mailpit

Mailpit intercepte tous les emails envoyés en local et les affiche dans une interface web. Aucun email ne part vraiment — tu peux tester tranquillement.

```
# Installation via Homebrew
brew install mailpit

# Lancer Mailpit
mailpit
```

```
# Interface disponible sur : http://localhost:8025
# SMTP local sur : localhost:1025
```

```
# Développement — Mailpit (emails interceptés localement)
EMAIL_HOST=localhost
EMAIL_PORT=1025
EMAIL_FROM=noreply-dev@socialseller.app
EMAIL_SECURE=false
```

```
# Staging — Mailtrap (service en ligne, emails capturés)
# EMAIL_HOST=sandbox.smtp.mailtrap.io
# EMAIL_PORT=2525
# EMAIL_FROM=noreply-staging@socialseller.app
# EMAIL_USER=xxxxx
# EMAIL_PASS=xxxxx
```

```
# Production — Resend ou SendGrid
# EMAIL_HOST=smtp.resend.com
# EMAIL_PORT=465
# EMAIL_FROM=noreply@socialseller.app
# EMAIL_API_KEY=re_xxxxxxxxxx
```

■ Pour la production, utilise Resend (resend.com) — simple, moderne, et s'intègre parfaitement avec Supabase. Gratuit jusqu'à 3000 emails/mois.

■ Ne mets JAMAIS les credentials SMTP de production dans ton .env.local. Configure-les uniquement dans les variables d'environnement de Railway/Supabase en production.

Partie 2 — Règles de Sécurité Fondamentales

2.1 — Gestion des secrets et variables d'environnement

C'est la règle n°1. Une clé API exposée dans du code public = compromission immédiate.

- Toutes les clés API, tokens, mots de passe dans des variables d'environnement — jamais en dur dans le code
- Ajoute .env, .env.local, .env.production au .gitignore AVANT le premier commit
- Utilise des noms de variables explicites : SUPABASE_SERVICE_ROLE_KEY (jamais 'key' ou 'secret')
- Différencie bien les clés Supabase : anon key (publique, front) vs service_role key (secrète, back uniquement)
- Rotate tes clés immédiatement si tu suspectes une fuite

■ La clé service_role de Supabase bypass toutes les politiques RLS. Elle ne doit JAMAIS être dans le code front-end ni dans un dépôt public.

2.2 — Multi-tenant : isolation des données (RLS)

L'isolation des tenants est la vulnérabilité la plus critique d'un SaaS multi-tenant. Un bug ici expose les données d'un client à un autre.

- Active Row Level Security (RLS) sur CHAQUE table sans exception

- Chaque policy RLS doit filtrer par `tenant_id = auth.jwt()->'tenant_id'`
- Teste l'isolation avec 2 comptes distincts avant chaque mise en production
- Ne fais jamais de requête sans filtre `tenant_id` côté serveur
- Journalise toutes les tentatives d'accès cross-tenant

```
-- Activer RLS sur la table
ALTER TABLE conversations ENABLE ROW LEVEL SECURITY;

-- Policy : un user ne voit que les conversations de son tenant
CREATE POLICY "tenant_isolation_conversations"
ON conversations
FOR ALL
USING (
tenant_id = (
SELECT tenant_id FROM users
WHERE id = auth.uid()
)
);

-- Policy super_admin : voit tout
CREATE POLICY "super_admin_all_conversations"
ON conversations
FOR ALL
USING (
EXISTS (
SELECT 1 FROM users
WHERE id = auth.uid() AND role = 'super_admin'
)
);
```

2.3 — Authentication & Sessions

- Utilise Supabase Auth — n'implémente jamais ton propre système d'authentification
- Tokens d'activation à usage unique : après utilisation, marque-les 'used' immédiatement
- Expiration des tokens d'activation : 48h maximum
- Limite les tentatives de connexion : 5 essais max, puis blocage 15 minutes (rate limiting)
- Sessions côté mobile : stocke les tokens avec expo-secure-store (chiffré), jamais en AsyncStorage
- Refresh tokens : laisse Supabase les gérer automatiquement
- Déconnexion : invalide le token côté serveur, pas seulement côté client

2.4 — Sécurité des APIs et Webhooks

- Vérifie la signature de chaque webhook entrant (WhatsApp, Facebook, TikTok signent leurs payloads)
- Utilise HTTPS exclusivement — jamais de HTTP en production
- Rate limiting sur tous tes endpoints : max 100 req/min par IP
- Valide et sanitise TOUTES les entrées utilisateur avant de les stocker en base
- N'expose jamais d'erreurs techniques détaillées à l'utilisateur (stack traces, noms de tables, etc.)
- CORS : liste blanche des origines autorisées uniquement

```
const crypto = require('crypto');

function verifyWhatsAppSignature(req, res, next) {
  const signature = req.headers['x-hub-signature-256'];
  if (!signature) return res.status(401).json({ error: 'No signature' });

  const expectedSignature = 'sha256=' + crypto
    .createHmac('sha256', process.env.WHATSAPP_APP_SECRET)
    .update(req.rawBody) // rawBody, pas le body parsé
    .digest('hex');

  if (!crypto.timingSafeEqual(
    Buffer.from(signature),
    Buffer.from(expectedSignature)
  )) {
    return res.status(401).json({ error: 'Invalid signature' });
  }
  next();
}
```

2.5 — Stockage et chiffrement

- Tokens d'accès réseaux sociaux (Facebook, TikTok) : chiffre-les en base avec pgcrypto
- Images uploadées : Supabase Storage avec politiques d'accès par tenant
- Ne stocke jamais les mots de passe en clair — Supabase Auth le gère avec bcrypt
- Logs : ne journalise jamais de données personnelles (numéros WhatsApp, noms clients) en clair dans les logs
- Backups Supabase : active les backups automatiques dès le départ (plan Pro)

2.6 — Sécurité mobile spécifique

- expo-secure-store pour tous les tokens sensibles (chiffré par le keychain iOS / Keystore Android)
- Certificate Pinning en production (empêche les attaques man-in-the-middle)
- Ne stocke aucune donnée sensible dans le code compilé (clés en dur dans le bundle JS)
- Désactive les logs de debug en production (console.log peut exposer des données)
- App Review : désactive l'accès debug/developer menu en production

Partie 3 — Instructions Sécurité à Donner à Claude Code

Claude Code génère du code fonctionnel mais pas toujours sécurisé par défaut. Ces instructions système à coller en début de session garantissent un code sécurisé dès la génération.

3.1 — Instruction système globale (colle-la à chaque nouvelle session Claude Code)

Tu es un expert en développement sécurisé d'applications SaaS multi-tenant.
Pour TOUT le code que tu génères sur ce projet, applique systématiquement ces règles :

SECRETS & VARIABLES D'ENVIRONNEMENT :

- Toutes les clés API, tokens, mots de passe dans process.env.XXX ou Expo.Constants
- Jamais de valeur sensible en dur dans le code
- Ajoute un commentaire // TODO: add to .env si tu dois mettre un placeholder

MULTI-TENANT & ISOLATION :

- Chaque requête base de données doit filtrer par tenant_id
- N'écris jamais de requête qui retourne des données sans filtre tenant
- En Supabase : utilise toujours le client avec la session user (jamais service_role côté front)

VALIDATION DES ENTRÉES :

- Valide et sanitise toutes les entrées utilisateur avant traitement
- Utilise zod pour la validation TypeScript
- Longueur max sur tous les champs texte
- Rejette les caractères spéciaux non attendus

AUTHENTIFICATION :

- Vérifie le token auth sur CHAQUE endpoint protégé
- N'expose jamais d'infos sensibles dans les messages d'erreur
- Utilise des messages d'erreur génériques côté client

GESTION D'ERREURS :

- Try/catch sur tous les appels async
- Log les erreurs côté serveur, message générique côté client
- Ne renvoie jamais de stack trace ou de détails techniques à l'utilisateur

HTTPS & COMMUNICATION :

- Utilise HTTPS pour tous les appels externes
- Vérifie les signatures des webhooks entrants
- Valide les origines CORS

Rappelle-moi si je te demande quelque chose qui viole ces règles.

3.2 — Prompt spécifique pour la génération de schéma BDD

Quand tu génères du SQL pour Supabase, applique ces règles de sécurité :

1. Active ALWAYS Row Level Security sur chaque table :

```
ALTER TABLE xxx ENABLE ROW LEVEL SECURITY;
```

2. Crée une policy d'isolation tenant pour chaque table :

- Les users voient uniquement les données de leur tenant_id
- Les super_admin voient tout
- Les agents voient uniquement leurs données assignées

3. Chiffre les colonnes sensibles avec pgcrypto :

- access_token, refresh_token, api_keys
- Exemple : pgp_sym_encrypt(token, current_setting('app.encryption_key'))

4. N'utilise jamais SELECT * dans les requêtes – liste explicitement les colonnes

5. Ajoute des contraintes CHECK sur les enums et valeurs critiques

6. Crée des index uniquement sur les colonnes nécessaires (tenant_id, status, created_at)
7. Ajoute updated_at avec trigger auto-update sur chaque table modifiable

3.3 — Prompt spécifique pour les endpoints API / webhooks

Pour chaque endpoint API ou webhook que tu génères :

1. AUTHENTIFICATION : vérifie le token en premier, avant toute logique métier
`const { user, error } = await supabase.auth.getUser(token)`
`if (error || !user) return res.status(401).json({ error: 'Unauthorized' })`
2. AUTORISATION : vérifie que l'utilisateur a le droit d'accéder à cette ressource (role check + tenant_id check)
3. VALIDATION : utilise zod pour valider le body de la requête
`const schema = z.object({ ... })`
`const result = schema.safeParse(req.body)`
`if (!result.success) return res.status(400).json({ error: 'Invalid input' })`
4. RATE LIMITING : applique un rate limit sur cet endpoint (max 100 req/min par IP pour les endpoints publics, max 1000 req/min par tenant pour les endpoints authentifiés)
5. WEBHOOKS EXTERNES : vérifie la signature cryptographique du payload avant tout traitement
6. RÉPONSES D'ERREUR : utilise des messages génériques
 - 401 : 'Unauthorized'
 - 403 : 'Forbidden'
 - 404 : 'Not found'
 - 500 : 'Internal server error'Jamais de détails techniques dans la réponse client

3.4 — Prompt spécifique pour le stockage mobile

Pour le stockage de données sensibles dans l'app Expo :

1. Utilise TOUJOURS expo-secure-store pour :
 - Tokens d'accès et refresh tokens Supabase
 - Clés de chiffrement localesJamais AsyncStorage pour ces données
2. Utilise AsyncStorage uniquement pour :
 - Préférences UI non sensibles (thème, langue)
 - Cache de données non sensibles
3. Désactive les logs en production :
`if (___DEV___) { console.log(...) }`
Jamais de console.log contenant des tokens ou données utilisateur

4. Variables d'environnement Expo :

- Préfixe EXPO_PUBLIC_ uniquement pour les valeurs non sensibles
- Les valeurs EXPO_PUBLIC_ sont visibles dans le bundle compilé
- Clés secrètes : uniquement dans le serveur backend, jamais dans l'app

5. Ne stocke pas de données personnelles clients en cache local (numéros WhatsApp, noms, historique de conversations)

Checklist Sécurité — À Valider Avant Chaque Mise en Production

- Variables d'environnement : aucune clé en dur dans le code
- .env.local dans .gitignore avant le premier commit git
- RLS activé sur toutes les tables Supabase
- Test isolation tenant : Merchant A ne voit pas les données de Merchant B
- Tokens d'activation : expiration 48h + usage unique vérifié
- Signatures webhooks WhatsApp/Facebook/TikTok vérifiées
- HTTPS uniquement sur tous les endpoints de production
- Messages d'erreur génériques côté client (pas de stack traces)
- expo-secure-store utilisé pour tous les tokens dans l'app mobile
- Pas de console.log de données sensibles en production
- Rate limiting actif sur les endpoints publics
- Clé service_role Supabase uniquement sur le backend, jamais dans l'app
- Emails de développement interceptés par Mailpit (pas de vrais envois en local)
- Emails de staging uniquement vers l'équipe interne
- Backups Supabase activés